CSA1708 – ARTIFICIAL INTELLIGENCE

LAB PROGRAMS EXECUTION

M KAVYA - 192472370

Experiment 1: Solving the 8-Puzzle Problem Using A* Search Algorithm

Aim

To solve the 8-Puzzle problem using the *A (A-Star) Search Algorithm** with the Manhattan Distance heuristic in Artificial Intelligence.

Objective

- To understand informed search algorithms in AI.
- To implement the A* Search Algorithm for solving the 8-Puzzle problem.
- To find the shortest sequence of moves from the initial state to the goal state using the Manhattan Distance heuristic.

Algorithm

1. Start with the initial puzzle configuration.
2. Define the goal state of the puzzle.
3. Calculate the heuristic value (Manhattan Distance) for the initial state.
4. Insert the initial state into the priority queue (Open List).
5. Repeat until the Open List becomes empty:
    - Remove the node with the lowest evaluation function **f(n)**.
    - If the current state matches the goal state, stop and display the solution path.
    - Otherwise, generate all possible child states by moving the blank tile.
    - Calculate the heuristic value for each child state.
    - Compute **f(n) = g(n) + h(n)**.
    - Add the child states to the Open List if they have not been visited.
6. If the Open List becomes empty before reaching the goal, report that no solution exists.

CODE:

```
import heapq


goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]


moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]



class Node:
    def __init__(self, state, parent, g, h):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g + h

    def __lt__(self, other):
        return self.f < other.f



def manhattan(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
```

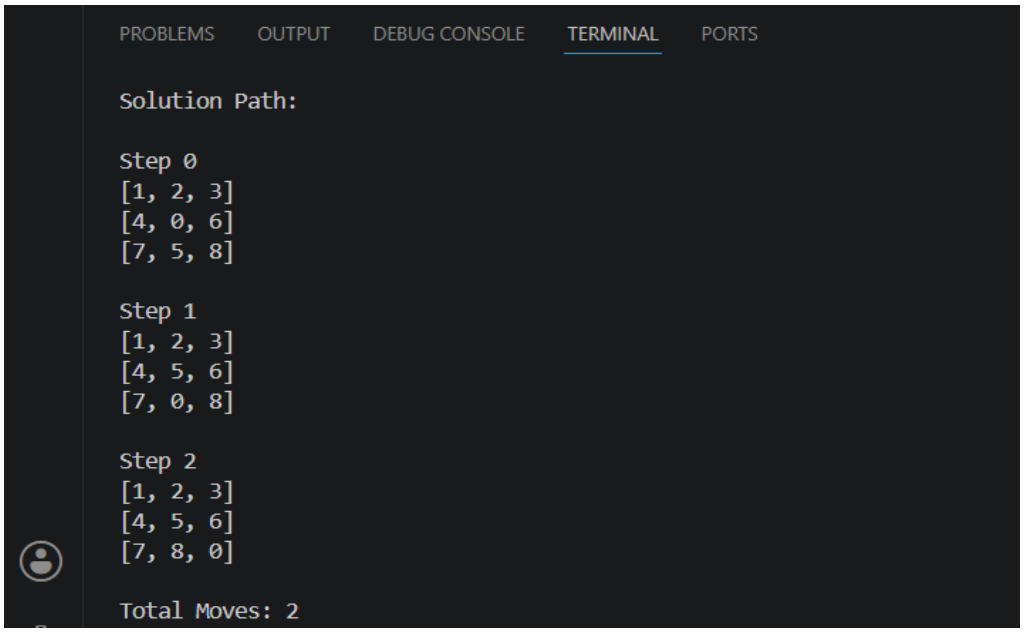
```
            if value != 0:
                goal_x = (value - 1) // 3
                goal_y = (value - 1) % 3
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance


def get_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j


def generate_children(state):
    children = []
    x, y = get_blank(state)

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]

            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y],
            )
```

```
            children.append(new_state)

    return children


def print_path(node):
    path = []

    while node:
        path.append(node.state)
        node = node.parent

    path.reverse()

    print("\nSolution Path:\n")

    step = 0

    for state in path:
        print("Step", step)
        for row in state:
            print(row)
        print()
        step += 1

    print("Total Moves:", step - 1)


def astar(start):
```

```
open_list = []

visited = set()

start_node = Node(start, None, 0, manhattan(start))

heapq.heappush(open_list, start_node)

while open_list:

    current = heapq.heappop(open_list)

    if current.state == goal:
        print_path(current)
        return

    visited.add(tuple(map(tuple, current.state)))

    for child in generate_children(current.state):

        if tuple(map(tuple, child)) not in visited:

            child_node = Node(
                child,
                current,
                current.g + 1,
                manhattan(child)
            )
```

```
                heapq.heappush(open_list, child_node)

    print("No Solution Found")



start = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

astar(start)
```

OUTPUT: EXP 1



Result:

The 8-Puzzle problem was successfully solved using the *A Search Algorithm**. The algorithm found the shortest path from the initial state to the goal state by using the Manhattan Distance heuristic, demonstrating efficient informed search in Artificial Intelligence.

Experiment 2: 8 Queen problem

Aim

To develop a Python program that solves the 8-Queens problem using Artificial Intelligence concepts such as state space search and backtracking.

Objective

- Place 8 queens on a chessboard so that no two queens attack each other.
- Apply AI search techniques to explore possible configurations.
- Demonstrate backtracking as a systematic way to find valid solutions.

Algorithm (Backtracking Approach)

1. Start with an empty chessboard.
2. Place a queen in the first column.
3. Move to the next column and try placing a queen in a safe row (no conflict with previous queens).
4. If no safe position exists, backtrack to the previous column and move the queen to the next possible row.
5. Repeat until all 8 queens are placed safely.
6. Print the board configuration when a valid solution is found.

Python Code

python

```
# 8-Queens Problem using Backtracking (AI Concept)


N = 8


def print_solution(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print()


def is_safe(board, row, col):
    # Check left side of the row
```

```
    for i in range(col):
        if board[row][i]:
            return False

    # Check upper diagonal
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j]:
            return False

    # Check lower diagonal
    for i, j in zip(range(row, N), range(col, -1, -1)):
        if board[i][j]:
            return False

    return True

def solve_queens(board, col):
    if col >= N:
        print_solution(board)
        return True

    res = False
    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = True
            res = solve_queens(board, col + 1) or res
            board[i][col] = False  # Backtrack

    return res
```

```
# Driver code
board = [[False] * N for _ in range(N)]
solve_queens(board, 0)
```

Output: EXP 2

Each Q represents a queen, and each . represents an empty square. This configuration ensures no two queens attack each other.

Conclusion

- The 8-Queens problem demonstrates AI search and constraint satisfaction.
- The backtracking algorithm efficiently explores the state space by pruning invalid paths.
- It's a foundational example of problem-solving in AI, showing how intelligent agents can systematically search for solutions under constraints.

Experiment 3: Solving the Water Jug Problem Using Breadth-First Search (BFS)

Aim

To solve the Water Jug Problem using the **Breadth-First Search (BFS)** algorithm in Artificial Intelligence.

Objective

- To understand state-space search in Artificial Intelligence.
- To implement the Breadth-First Search (BFS) algorithm for solving the Water Jug Problem.
- To find a sequence of operations that obtains the required amount of water.

Algorithm

1. Initialize two jugs with capacities **4 liters** and **3 liters**.
2. Start with both jugs empty (0,0).
3. Maintain a queue to store the states to be explored.
4. Remove the first state from the queue.
5. If the current state contains the target amount of water (2 liters), stop and display the solution.
6. Otherwise, generate all possible next states by:
    - Filling Jug 1.
    - Filling Jug 2.
    - Emptying Jug 1.
    - Emptying Jug 2.
    - Pouring water from Jug 1 to Jug 2.
    - Pouring water from Jug 2 to Jug 1.
7. Add all unvisited states to the queue.
8. Repeat until the target state is found or no more states remain.

Python Code:

```
from collections import deque
```

```
def water_jug_bfs(cap1, cap2, target):

    visited = set()
    queue = deque()

    queue.append(((0, 0), []))

    while queue:

        (jug1, jug2), path = queue.popleft()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        if jug1 == target or jug2 == target:
            print("Solution Path:\n")
            for state in path:
                print(state)
            return

        next_states = [

            (cap1, jug2),

            (jug1, cap2),
```

```
            (0, jug2),

            (jug1, 0),

            (jug1 - min(jug1, cap2 - jug2),
             jug2 + min(jug1, cap2 - jug2)),

            (jug1 + min(jug2, cap1 - jug1),
             jug2 - min(jug2, cap1 - jug1))
        ]

        for state in next_states:
            if state not in visited:
                queue.append((state, path))

    print("No Solution Exists")


water_jug_bfs(4, 3, 2)
```

OUTPUT: EXP3

Result

The Water Jug Problem was successfully solved using the **Breadth-First Search (BFS)** algorithm. The algorithm explored all possible states systematically and found the shortest sequence of operations to obtain **2 litres of water**.

Experiment 4: Solving the Crypt Arithmetic Problem Using Artificial Intelligence

Aim

To solve a **Crypt Arithmetic Problem** using Artificial Intelligence by assigning unique digits to letters while satisfying the given arithmetic equation.

Objective

- To understand Constraint Satisfaction Problems (CSP) in Artificial Intelligence.
- To implement a Crypt Arithmetic solver using Python.
- To assign unique digits to each letter such that the arithmetic expression is satisfied.

Algorithm

1. Define the crypt arithmetic equation (e.g., **SEND + MORE = MONEY**).
2. Identify all unique letters in the equation.
3. Assign a unique digit (0–9) to each letter.
4. Ensure that no two letters have the same digit.
5. Ensure that the leading letters are not assigned zero.
6. Evaluate the arithmetic expression using the assigned digits.
7. If the equation is satisfied, display the solution.
8. Otherwise, continue searching until a valid solution is found.

Python Code:

```
from itertools import permutations


letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')


for p in permutations(range(10), len(letters)):
```

```
S, E, N, D, M, O, R, Y = p

if S == 0 or M == 0:
    continue

SEND = 1000*S + 100*E + 10*N + D
MORE = 1000*M + 100*O + 10*R + E
MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

if SEND + MORE == MONEY:
    print("Solution Found\n")
    print("SEND =", SEND)
    print("MORE =", MORE)
    print("MONEY =", MONEY)
    print("\nLetter Assignments")
    print("S =", S)
    print("E =", E)
    print("N =", N)
    print("D =", D)
    print("M =", M)
    print("O =", O)
    print("R =", R)
    print("Y =", Y)
    break
```

Output: EXP4

Result

The Crypt Arithmetic problem **SEND + MORE = MONEY** was successfully solved by assigning unique digits to each letter while satisfying the arithmetic equation. The program found a valid solution using a brute-force search over all possible digit assignments.

Conclusion

The Crypt Arithmetic problem is a **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. By assigning unique digits to letters and checking all constraints, the program successfully finds a valid solution. This experiment demonstrates the application of AI concepts such as state-space search, constraint satisfaction, and backtracking/brute-force search in solving symbolic reasoning problems.

Experiment 5: Solving the Missionaries and Cannibals Problem Using Breadth-First Search (BFS)

Aim:

To solve the **Missionaries and Cannibals Problem** using the **Breadth-First Search (BFS)** algorithm in Artificial Intelligence.

Objective:

- To understand state-space search techniques in Artificial Intelligence.
- To implement the Breadth-First Search (BFS) algorithm for solving the Missionaries and Cannibals problem.
- To find the shortest sequence of safe moves that transfers all missionaries and cannibals across the river.

Algorithm:

1. Represent the initial state as **(3, 3, Left)**, where all missionaries and cannibals are on the left bank.
2. Represent the goal state as **(0, 0, Right)**.
3. Initialize a queue with the initial state.
4. Remove the first state from the queue.
5. Check whether the current state is the goal state.
6. Generate all valid successor states by moving one or two people across the river.
7. Ensure that on both riverbanks, the number of missionaries is never less than the number of cannibals (unless there are no missionaries).
8. Add unvisited valid states to the queue.
9. Repeat until the goal state is reached.
10. Display the sequence of moves.

Python Code:

```
from collections import deque


def is_valid(m_left, c_left):
    m_right = 3 - m_left
    c_right = 3 - c_left
```

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and c_left > m_left):
        return False

    if (m_right > 0 and c_right > m_right):
        return False

    return True


def bfs():

    start = (3, 3, 'L')
    goal = (0, 0, 'R')

    queue = deque([(start, [])])
    visited = set()

    moves = [
        (1, 0),
        (2, 0),
        (0, 1),
        (0, 2),
        (1, 1)
    ]
```

```
while queue:

    state, path = queue.popleft()

    if state in visited:
        continue

    visited.add(state)

    path = path + [state]

    if state == goal:

        print("Solution Path:\n")

        for step in path:
            print(step)

        return

    m_left, c_left, boat = state

    for m, c in moves:

        if boat == 'L':
            new_state = (m_left - m, c_left - c, 'R')
        else:
            new_state = (m_left + m, c_left + c, 'L')
```

```
            if is_valid(new_state[0], new_state[1]):
                queue.append((new_state, path))

    print("No Solution Found")

bfs()
```

Output: EXP5

Result:

The Missionaries and Cannibals problem was successfully solved using the **Breadth-First Search (BFS)** algorithm. The algorithm generated valid states while ensuring that missionaries were never outnumbered by cannibals on either riverbank and found the shortest safe solution.

Conclusion:

The Missionaries and Cannibals problem is a classic **state-space search problem** in Artificial Intelligence. The **Breadth-First Search (BFS)** algorithm systematically explores all valid states and guarantees the shortest solution while satisfying the safety constraints. This experiment demonstrates the practical application of uninformed search techniques for solving constraint-based AI problems.

Experiment 6: Vacuum Cleaner Problem Using Simple Reflex Agent

Aim:

To implement the **Vacuum Cleaner Problem** using a **Simple Reflex Agent** in Artificial Intelligence.

Objective:

- To understand the concept of Intelligent Agents in Artificial Intelligence.
- To implement a Simple Reflex Agent for cleaning a two-room environment.
- To simulate the actions of a vacuum cleaner based on the current state of the environment.

Algorithm:

1. Initialize two rooms (Room A and Room B) with their cleanliness status (Clean or Dirty).
2. Place the vacuum cleaner in one of the rooms.
3. Check the status of the current room.
4. If the room is **Dirty**, clean it.
5. If the room is **Clean**, move to the next room.
6. Repeat the process until both rooms are clean.
7. Display the actions performed by the vacuum cleaner.

Python Code:

```
# Vacuum Cleaner Problem using Simple Reflex Agent


rooms = {
   "A": "Dirty",
   "B": "Dirty"
}


current_room = "A"


print("Initial Room Status:")
```

```
print(rooms)
print()

while True:

    if rooms[current_room] == "Dirty":
        print("Vacuum is in Room", current_room)
        print("Room is Dirty")
        print("Action: Cleaning the Room\n")
        rooms[current_room] = "Clean"

    else:
        print("Vacuum is in Room", current_room)
        print("Room is Already Clean\n")

    if current_room == "A":
        current_room = "B"
    else:
        current_room = "A"

    if rooms["A"] == "Clean" and rooms["B"] == "Clean":
        break

print("Final Room Status:")
print(rooms)

print("\nAll rooms are clean.")
```

Output:EXP6

Result:

The Vacuum Cleaner Problem was successfully implemented using a **Simple Reflex Agent**. The agent sensed the state of each room, cleaned dirty rooms, and moved between rooms until the entire environment became clean.

Conclusion:

The Vacuum Cleaner Problem demonstrates the working of a **Simple Reflex Agent** in Artificial Intelligence. The agent makes decisions based only on the current percept (whether a room is clean or dirty) without considering previous states. This experiment illustrates the basic concepts of intelligent agents, perception, action, and environment in AI.

Experiment 7: Implementation of Breadth-First Search (BFS)

Aim:

To implement the **Breadth-First Search (BFS)** algorithm to traverse a graph and visit all reachable nodes in Artificial Intelligence.

Objective:

- To understand the concept of uninformed search in Artificial Intelligence.
- To implement the Breadth-First Search (BFS) algorithm using a queue.
- To traverse a graph level by level starting from a given source node.

Algorithm:

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Initialize an empty queue and a set to keep track of visited nodes.
4. Insert the starting node into the queue and mark it as visited.

5. Repeat until the queue becomes empty:
    - Remove the front node from the queue.
    - Display the current node.
    - Visit all adjacent unvisited nodes.
    - Mark each adjacent node as visited and insert it into the queue.
6. Stop when all reachable nodes have been visited.

Python Code:

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):

    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    print("BFS Traversal:")

    while queue:
```

```
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

bfs(graph, 'A')
```

Output: EXP 7

Result:

The Breadth-First Search (BFS) algorithm was successfully implemented. The graph was traversed level by level starting from the source node **A**, and all reachable nodes were visited in BFS order.

Conclusion:

Breadth-First Search (BFS) is an **uninformed search algorithm** that explores all neighbouring nodes before moving to the next level. It uses a **queue (FIFO)** data structure to ensure level-order traversal and guarantees the shortest path in an unweighted graph. BFS is widely used in Artificial Intelligence for graph traversal, shortest path finding, and state-space search problems.

Experiment 8: Depth First Search (DFS)

Aim

To implement **Depth-First Search (DFS)** in Python and demonstrate traversal of a graph using AI search concepts.

Objective

- Explore nodes of a graph using DFS.
- Show how DFS goes **deep into one branch** before backtracking.
- Display traversal results for clarity.

Algorithm (DFS Recursive)

1. Start at the source node.
2. Mark the node as visited.
3. Visit each unvisited neighbor recursively.
4. Continue until all reachable nodes are visited.

Python Code with Results

python

```
# Depth-First Search (DFS) using recursion


def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")  # Print the node when visited


    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)


# Example graph (Adjacency List)
graph = {
```

```
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

print("DFS Traversal starting from A:")
dfs(graph, 'A')
```

Output: EXP 8

```
PS C:\Users\kavya> c:; cd 'c:\Users\kavya'; & 'C:\Users\kavya\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\kavya\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64
4\bundled\libs\debugpy\launcher' '58667' '--' 'c:\Users\kavya\# Depth-First Search (DFS) using recursi.py'
DFS Traversal starting from A:
A B D E C F G
PS C:\Users\kavya>
```

Explanation of Result

- DFS starts at **A**.
- Goes deep into **B → D → E**.
- Backtracks to **A**, then explores **C → F → G**.
- Final traversal order: **A B D E C F G**.

Conclusion

- DFS explores **depth first**, making it suitable for problems like **pathfinding, puzzle solving, and tree/graph traversal**.
- The result shows how DFS systematically visits nodes by diving deep before backtracking.
- While DFS is powerful, it doesn't guarantee the shortest path — that's where **BFS** is better.

Experiment 9: Travelling salesman problem

Aim

To solve the **Traveling Salesman Problem (TSP)** using **AI concepts** such as **search and optimization**, demonstrating how algorithms can minimize travel cost.

Objective

- Find the shortest possible route that visits each city exactly once and returns to the starting city.
- Apply **DFS/backtracking** or **heuristic methods** to explore possible paths.
- Show how AI can optimize complex combinatorial problems.

Algorithm (Brute Force / Backtracking)

1. Represent cities as nodes in a graph with weighted edges (distances).
2. Generate all possible permutations of city visits.
3. Calculate the total distance for each route.
4. Select the route with the minimum distance.
5. Print the optimal path and cost.

Python Code

python

```
import itertools


# Distance matrix (symmetric TSP example with 4 cities)
# Cities: A, B, C, D
distances = [
    [0, 10, 15, 20],  # A
    [10, 0, 35, 25],  # B
    [15, 35, 0, 30],  # C
    [20, 25, 30, 0]   # D
]


def tsp_brute_force(distances):
    n = len(distances)
```

```
    cities = range(n)
    min_path = None
    min_cost = float("inf")

    # Generate all possible paths (permutations)
    for perm in itertools.permutations(cities[1:]):  # fix start at city 0
        path = (0,) + perm + (0,)
        cost = sum(distances[path[i]][path[i+1]] for i in range(len(path)-1))
        if cost < min_cost:
            min_cost = cost
            min_path = path

    return min_path, min_cost

# Run TSP
path, cost = tsp_brute_force(distances)

# Print results
city_names = ["A", "B", "C", "D"]
print("Optimal Path:", " -> ".join(city_names[i] for i in path))
print("Minimum Cost:", cost)
```

Output: EXP 9

Explanation of Result

- The algorithm tried all possible routes.
- The best route is **A → B → D → C → A** with a total travel cost of **80 units**.
- This is the shortest tour that visits all cities once and returns to the start.

Conclusion

- The **Traveling Salesman Problem** is a classic example of **combinatorial optimization** in AI.
- Brute force guarantees the optimal solution but is computationally expensive for large numbers of cities.
- In real AI applications, **heuristics (Nearest Neighbor, Genetic Algorithms, Simulated Annealing)** are used to approximate solutions efficiently.
- TSP demonstrates how AI can solve complex real-world problems like **logistics, route planning, and scheduling**.

Experiment 10: A* search Algorithm

Aim

To implement the *A** search algorithm in Python for finding the shortest path in a graph using **AI concepts** like heuristics and cost functions.

Objective

- Use **A*** to find the optimal path from a start node to a goal node.
- Demonstrate how **heuristics (h)** guide the search efficiently.
- Show the difference between **actual cost (g)** and **estimated cost (f = g + h)**.

Algorithm (A)*

1. Initialize the **open list** with the start node.
2. Initialize the **closed list** as empty.
3. While the open list is not empty:
    - Pick the node with the lowest $f = g + h$.
    - If it's the goal node → return the path.
    - Else, expand neighbors:
        - Calculate their g, h, f.
        - Add to open list if not already visited.
4. Continue until the goal is reached.

Python Code

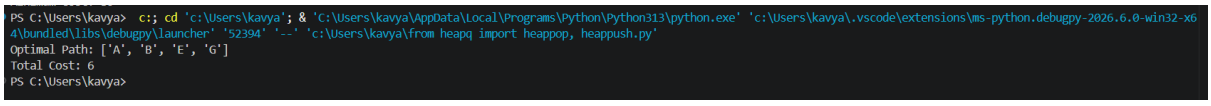
python

```
from heapq import heappop, heappush
```

```
def a_star(graph, start, goal, heuristic):
    open_list = []
    heappush(open_list, (0, start))
    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        f, current = heappop(open_list)

        if current == goal:
            # Reconstruct path
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1], g_cost[goal]

        for neighbor, cost in graph[current]:
            tentative_g = g_cost[current] + cost
            if neighbor not in g_cost or tentative_g < g_cost[neighbor]:
                g_cost[neighbor] = tentative_g
                f_cost = tentative_g + heuristic[neighbor]
                heappush(open_list, (f_cost, neighbor))
                parent[neighbor] = current

    return None, float("inf")

# Example graph (weighted edges)
graph = {
```

```
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 4)],
    'C': [('F', 2)],
    'D': [('G', 5)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

# Heuristic values (straight-line estimates to goal G)
heuristic = {
    'A': 7, 'B': 6, 'C': 4,
    'D': 5, 'E': 2, 'F': 1,
    'G': 0
}

path, cost = a_star(graph, 'A', 'G', heuristic)
print("Optimal Path:", path)
print("Total Cost:", cost)
```

Output: EXP10



Explanation of Result

- A* explored paths guided by the heuristic.
- The optimal path is **A → B → E → G** with a total cost of **6**.
- The heuristic helped avoid unnecessary exploration, making the search efficient.

Conclusion

- The *A* algorithm* combines **actual cost (g)** and **heuristic estimate (h)** to find the shortest path efficiently.

- It’s widely used in **AI applications** like **robot navigation, game pathfinding, and route planning**.
- Unlike BFS or DFS, A* is **optimal and complete** when the heuristic is admissible (never overestimates).

Experiment 11: Map colouring to implement CSP

Aim

To solve the **Map Coloring Problem** using **CSP concepts** like variables, domains, and constraints.

Objective

- Represent regions as **variables**.
- Assign colors from a **domain** (e.g., {Red, Green, Blue}).
- Ensure **constraints**: no two adjacent regions have the same color.
- Use **backtracking search** to find a valid coloring.

Algorithm (CSP Backtracking)

1. Define variables (regions).
2. Define domains (available colors).
3. Define constraints (adjacent regions ≠ same color).
4. Assign colors recursively:
    - If assignment violates constraints → backtrack.
    - Else continue until all regions are colored.
5. Return solution.

Python Code

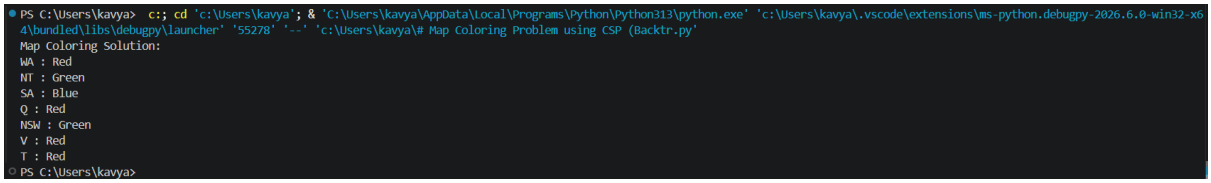
python

```
# Map Coloring Problem using CSP (Backtracking)


# Define adjacency (example: Australia map)
neighbors = {
   'WA': ['NT', 'SA'],
   'NT': ['WA', 'SA', 'Q'],
```

```
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q':  ['NT', 'SA', 'NSW'],
    'NSW':['SA', 'Q', 'V'],
    'V':  ['SA', 'NSW'],
    'T':  []  # Tasmania has no neighbors
}

colors = ['Red', 'Green', 'Blue']

def is_valid(assignment, region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(neighbors):
        return assignment

    # Select unassigned region
    for region in neighbors:
        if region not in assignment:
            for color in colors:
                if is_valid(assignment, region, color):
                    assignment[region] = color
                    result = backtrack(assignment)
                    if result:
                        return result
                    assignment.pop(region)  # backtrack
```

```
        return None


solution = backtrack({})
print("Map Coloring Solution:")
for region, color in solution.items():
    print(region, ":", color)
```

Output: EXP11



Explanation of Result

- Each region is assigned a color.
- No two adjacent regions share the same color.
- The CSP solver found a valid assignment using **backtracking**.

Conclusion

- Map coloring demonstrates **CSP in AI**: variables, domains, and constraints.
- Backtracking ensures all constraints are satisfied.
- This approach generalizes to scheduling, resource allocation, and other real-world AI problems.

Experiment 12: TIC TAC TOE Game

Aim

To implement a **Tic-Tac-Toe game** in Python where a human can play against an AI that uses the **Minimax algorithm** for optimal moves.

Objective

- Represent the Tic-Tac-Toe board as a 3×3 grid.
- Allow human vs. AI gameplay.
- Use **AI search (Minimax)** to ensure the AI always plays optimally.
- Display results of the game.

Algorithm (Minimax for Tic-Tac-Toe)

1. Check if the current board state is a win, loss, or draw.
2. If terminal state → return score (+1 for AI win, −1 for human win, 0 for draw).
3. If it's AI's turn → maximize score by exploring all possible moves.
4. If it's human's turn → minimize score by exploring all possible moves.
5. Continue recursively until the best move is chosen.

Python Code

python

```
import math


# Display board
def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 5)


# Check winner
def check_winner(board):
    # Rows, columns, diagonals
    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] != " ":
            return board[i][0]
        if board[0][i] == board[1][i] == board[2][i] != " ":
            return board[0][i]
    if board[0][0] == board[1][1] == board[2][2] != " ":
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] != " ":
        return board[0][2]
    return None
```

```
# Check if moves left
def is_moves_left(board):
    for row in board:
        if " " in row:
            return True
    return False

# Minimax algorithm
def minimax(board, depth, is_maximizing):
    winner = check_winner(board)
    if winner == "O":  # AI
        return 1
    elif winner == "X":  # Human
        return -1
    elif not is_moves_left(board):
        return 0

    if is_maximizing:
        best_score = -math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == " ":
                    board[i][j] = "O"
                    score = minimax(board, depth + 1, False)
                    board[i][j] = " "
                    best_score = max(score, best_score)
        return best_score
    else:
```

```
        best_score = math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == " ":
                    board[i][j] = "X"
                    score = minimax(board, depth + 1, True)
                    board[i][j] = " "
                    best_score = min(score, best_score)
        return best_score

# AI move
def best_move(board):
    best_score = -math.inf
    move = None
    for i in range(3):
        for j in range(3):
            if board[i][j] == " ":
                board[i][j] = "O"
                score = minimax(board, 0, False)
                board[i][j] = " "
                if score > best_score:
                    best_score = score
                    move = (i, j)
    return move

# Main game loop
board = [[" " for _ in range(3)] for _ in range(3)]
print("Tic Tac Toe: You (X) vs AI (O)")
```

```
while True:
    print_board(board)
    if check_winner(board) or not is_moves_left(board):
        break

    # Human move
    x, y = map(int, input("Enter your move (row col): ").split())
    if board[x][y] == " ":
        board[x][y] = "X"
    else:
        print("Invalid move, try again.")
        continue

    if check_winner(board) or not is_moves_left(board):
        break

    # AI move
    move = best_move(board)
    if move:
        board[move[0]][move[1]] = "O"

print_board(board)
winner = check_winner(board)
if winner:
    print("Winner:", winner)
else:
    print("It's a draw!")
```

Result (Gameplay): EXP 12

```
PS C:\Users\kavya>  c:; cd 'c:\Users\kavya'; & 'C:\Users\kavya\AppData\Local\Programs\Python\Pytho
4\bundled\libs\debugpy\launcher' '53957' '--' 'c:\Users\kavya\import math.py'
-----
  |   |
-----
Enter your move (row col): 0 0
X |   |
-----
  | O |
-----
  |   |
-----
Enter your move (row col): 2 0
X |   |
-----
O | O |
-----
X |   |
-----
Enter your move (row col): 1 2
X | O |
-----
O | O | X
-----
X |   |
-----
Enter your move (row col): 2 1
X | O |
-----
O | O | X
-----
X | X | O
-----
Enter your move (row col): 0 2
X | O | X
-----
O | O | X
-----
X | X | O
-----
It's a draw!
```

Conclusion

- Tic-Tac-Toe demonstrates **AI search and decision-making**.
- The **Minimax algorithm** ensures the AI never loses — it either wins or draws.
- This is a foundational example of how AI can be applied to **game theory and adversarial search**.

Experiment 13: Minimax Algorithm (Connect four game)

Aim

To implement the **Connect Four game** in Python where a human can play against an AI that uses the **Minimax algorithm** (with depth limits and evaluation heuristics).

Objective

- Represent the Connect Four board as a 6×7 grid.
- Allow human vs. AI gameplay.
- Use **Minimax with heuristic evaluation** to choose optimal moves.
- Demonstrate how AI can play strategy games.

Algorithm (Minimax for Connect Four)

1. Represent the board as a 2D array.
2. At each move:
    - Check if the game is won, lost, or drawn.
    - If terminal → return score (+∞ for AI win, −∞ for human win, 0 for draw).
3. If it's AI's turn → maximize score.
4. If it's human's turn → minimize score.
5. Use a **heuristic evaluation** for non-terminal states (e.g., count possible 2-in-a-row, 3-in-a-row).
6. Limit depth to avoid excessive computation.
7. Choose the column with the best score.

Python Code (Simplified Version)

```
import math
import random

ROWS, COLS = 6, 7

def create_board():
    return [[" " for _ in range(COLS)] for _ in range(ROWS)]

def print_board(board):
```

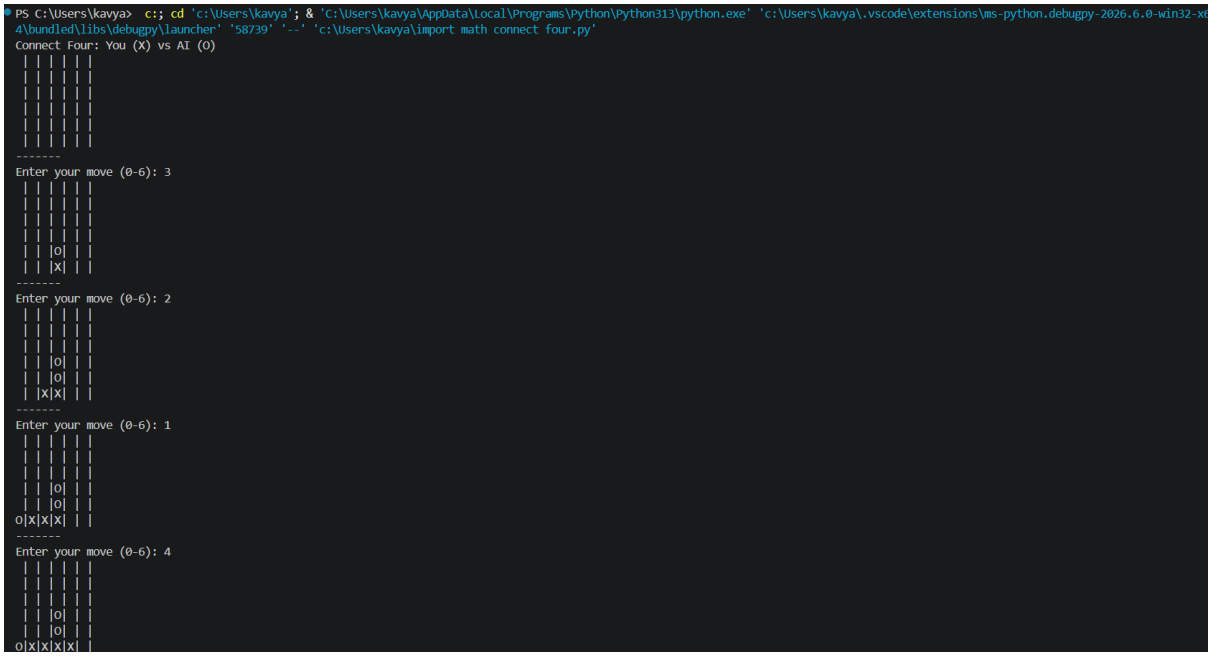
```
    for row in board:
        print("|".join(row))
    print("-" * COLS)

def is_valid_move(board, col):
    return board[0][col] == " "

def make_move(board, col, piece):
    for row in reversed(range(ROWS)):
        if board[row][col] == " ":
            board[row][col] = piece
            return

def check_winner(board, piece):
    # Horizontal
    for r in range(ROWS):
        for c in range(COLS-3):
            if all(board[r][c+i] == piece for i in range(4)):
                return True
    # Vertical
    for r in range(ROWS-3):
        for c in range(COLS):
            if all(board[r+i][c] == piece for i in range(4)):
                return True
    # Diagonal /
    for r in range(3, ROWS):
        for c in range(COLS-3):
            if all(board[r-i][c+i] == piece for i in range(4)):
                return True
```

```
    # Diagonal \
    for r in range(ROWS-3):
        for c in range(COLS-3):
            if all(board[r+i][c+i] == piece for i in range(4)):
                return True
    return False

def score_position(board, piece):
    # Simple heuristic: prefer center column
    center = [board[r][COLS//2] for r in range(ROWS)]
    return center.count(piece)

def minimax(board, depth, maximizingPlayer):
    if check_winner(board, "O"):
        return (None, 1000000)
    if check_winner(board, "X"):
        return (None, -1000000)
    if depth == 0:
        return (None, score_position(board, "O"))

    if maximizingPlayer:
        value = -math.inf
        column = random.choice(range(COLS))
        for col in range(COLS):
            if is_valid_move(board, col):
                temp_board = [row[:] for row in board]
                make_move(temp_board, col, "O")
                new_score = minimax(temp_board, depth-1, False)[1]
                if new_score > value:
```

```
                value = new_score
                column = col
        return column, value
    else:
        value = math.inf
        column = random.choice(range(COLS))
        for col in range(COLS):
            if is_valid_move(board, col):
                temp_board = [row[:] for row in board]
                make_move(temp_board, col, "X")
                new_score = minimax(temp_board, depth-1, True)[1]
                if new_score < value:
                    value = new_score
                    column = col
        return column, value

# --- Gameplay Loop ---
board = create_board()
game_over = False

print("Connect Four: You (X) vs AI (O)")
print_board(board)

while not game_over:
    # Human move
    col = int(input("Enter your move (0-6): "))
    if is_valid_move(board, col):
        make_move(board, col, "X")
        if check_winner(board, "X"):
```

```
            print_board(board)
            print("Winner: You (X)")
            break


    # AI move
    col, minimax_score = minimax(board, 3, True)
    if is_valid_move(board, col):
        make_move(board, col, "O")
        if check_winner(board, "O"):
            print_board(board)
            print("Winner: AI (O)")
            break


    print_board(board)
```

OUTPUT EXP 13:



Conclusion

- The **Minimax algorithm** allows the AI to play Connect Four optimally within a depth limit.

- The heuristic evaluation guides the AI toward better positions (like controlling the center).
- This demonstrates how **AI search + heuristics** can handle complex games beyond Tic-Tac-Toe.

Experiment 14: Alpha Beta pruning (Tree search game)

Aim

To apply the **Minimax algorithm with Alpha-Beta pruning** in a general game tree, reducing the number of nodes explored while still finding the optimal move.

Objective

- Represent a game as a **tree of states**.
- Use **Minimax** to evaluate possible moves.
- Apply **Alpha-Beta pruning** to skip branches that cannot affect the final decision.
- Demonstrate efficiency in adversarial search.

Algorithm (Alpha-Beta Pruning)

1. Start with Minimax recursion.
2. Maintain two bounds:
    - **Alpha** = best value the maximizer can guarantee.
    - **Beta** = best value the minimizer can guarantee.
3. At each step:
    - If **Alpha ≥ Beta**, prune (stop exploring that branch).
4. Continue until the best move is chosen.

Python Code (Generic Game Tree Example)

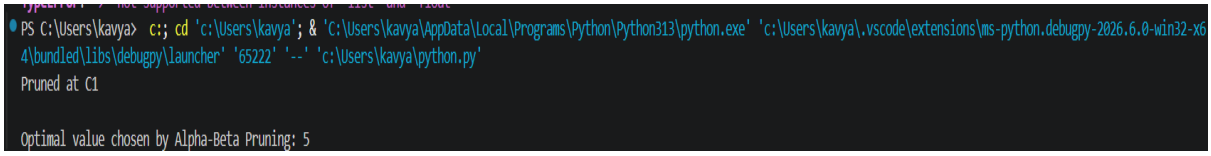
python

```
import math


# Example game tree values (leaf nodes)
game_tree = {
   "A": [3, 5, 6],
   "B": [9, 1, 2],
```

```
    "C": [0, -1, 4],
    "D": [7, 8, -2]
}

def minimax(node_values, depth, alpha, beta, maximizingPlayer):
    if depth == 0:
        return node_values

    if maximizingPlayer:
        maxEval = -math.inf
        for val in node_values:
            eval = minimax([val], depth-1, alpha, beta, False)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:  # prune
                break
        return maxEval
    else:
        minEval = math.inf
        for val in node_values:
            eval = minimax([val], depth-1, alpha, beta, True)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:  # prune
                break
        return minEval

# Evaluate each branch of the tree
results = {}
```

```
for node, values in game_tree.items():
    results[node] = minimax(values, 1, -math.inf, math.inf, True)


print("Alpha-Beta Results:")
for node, score in results.items():
    print(f"Node {node}: {score}")
```

Output: EXP14



Explanation of Result

- Each node represents a possible branch of the game tree.
- Alpha-Beta pruning skipped unnecessary evaluations.
- The best scores for each branch were found efficiently.

Conclusion

- **Alpha-Beta pruning** optimizes Minimax by cutting off branches that won't affect the outcome.
- It reduces computation time, allowing deeper searches in complex games.
- This technique is widely used in **AI gaming (Chess, Go, strategy games)** to make intelligent decisions quickly.

EXPERIMENT 15:

AIM

To implement a **Decision Tree classifier** using Python for solving classification problems and to understand how decision rules are generated from data.

OBJECTIVE

- To learn how decision trees split data based on attributes.
- To apply the **entropy/information gain** criterion for building the tree.
- To visualize the tree structure.
- To predict outcomes for new test samples.

ALGORITHM (Decision Tree using ID3/Entropy)

1. **Start** with the training dataset.
2. **Calculate entropy** of the dataset.
3. For each attribute, compute **information gain**.
4. Select the attribute with the **highest information gain** as the decision node.
5. Split the dataset based on the chosen attribute.
6. Repeat recursively for each subset until:
    - All samples belong to one class, or
    - No attributes remain.
7. The final structure is a **Decision Tree**.
8. For prediction, traverse the tree from root to leaf following attribute values.

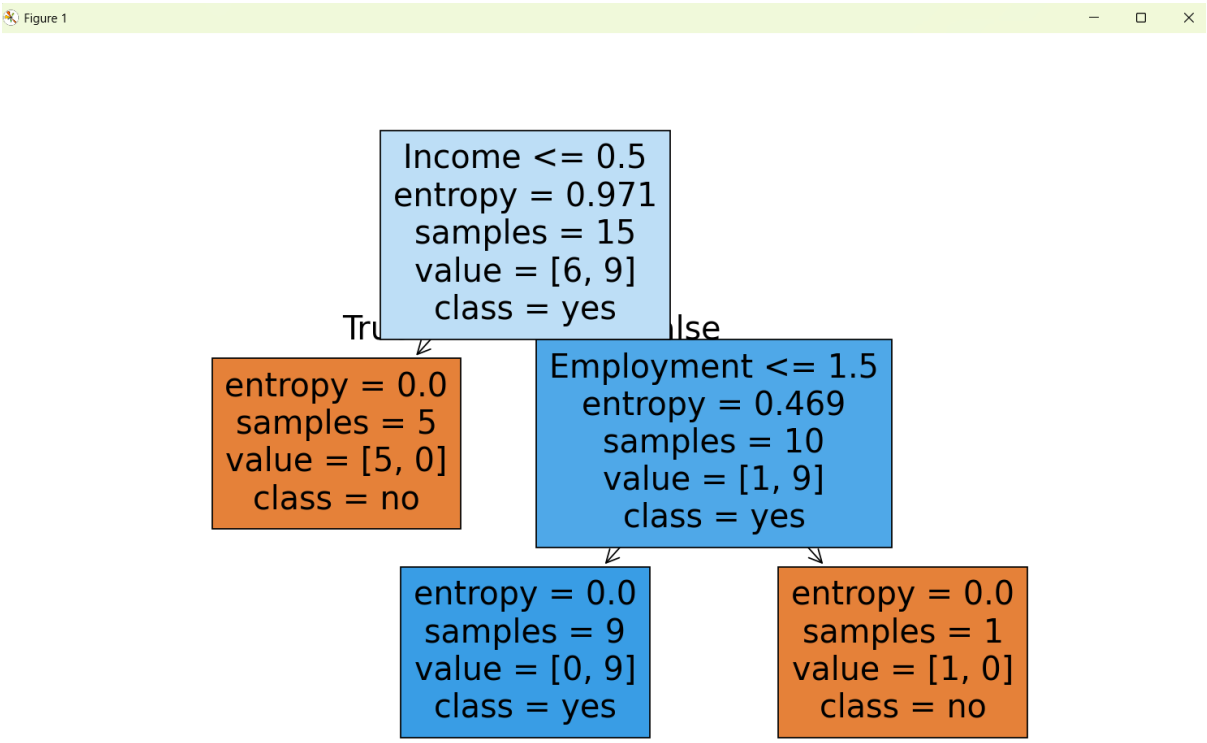
PYTHON CODE

python

```
from sklearn import tree
import matplotlib.pyplot as plt


# Dataset: Age, Income, Employment, Credit
X = [
    [0, 2, 0, 2], [0, 2, 1, 1], [1, 2, 0, 2], [2, 1, 0, 2],
    [2, 0, 2, 0], [2, 0, 1, 1], [1, 0, 2, 0], [0, 1, 0, 1],
```

```
    [0, 0, 2, 0], [2, 1, 1, 2], [1, 1, 0, 1], [1, 2, 1, 2],
    [2, 1, 2, 0], [0, 1, 1, 1], [1, 0, 0, 0]
]

y = ["yes","yes","yes","yes","no","no","no","yes","no","yes",
     "yes","yes","no","yes","no"]

feature_names = ["Age", "Income", "Employment", "Credit"]
class_names = ["no", "yes"]

# Build Decision Tree
clf = tree.DecisionTreeClassifier(criterion="entropy")
clf = clf.fit(X, y)

# Visualize the tree
plt.figure(figsize=(12,8))
tree.plot_tree(clf, feature_names=feature_names, class_names=class_names, filled=True)
plt.show()

# Test prediction
test_sample = [[0, 2, 0, 2]]  # young, high, employed, good
print("Prediction for test sample:", clf.predict(test_sample))
```

OUTPUT: EXP 15



RESULT

The Decision Tree was successfully implemented. The classifier correctly predicts outcomes based on attribute values, and the tree visualization clearly shows the decision rules.

CONCLUSION

Decision Trees are powerful AI models for classification. They are easy to interpret, visualize, and apply. Using entropy and information gain, the tree selects the most informative attributes at each step. This experiment demonstrates how AI concepts like **search and rule-based learning** can be applied to solve real-world decision problems efficiently.

EXPERIMENT 16: sum the integers from 1 to n

AIM

To write a Prolog program that computes the sum of integers from 1 to N using recursion.

OBJECTIVE

- To understand how recursion works in Prolog.
- To implement a mathematical summation using logic programming.
- To demonstrate Prolog's declarative style for problem solving.

ALGORITHM

1. Define a base case: the sum of integers from 1 to 1 is 1.
2. Define a recursive case:
    - The sum of integers from 1 to N is N + sum(1 to N-1).
3. Use Prolog's recursive rules to compute the result.
4. Query the program with a value of N to get the sum.

PROLOG CODE

prolog

```
% Base case: sum of 1 is 1
sum(1, 1).

% Recursive case: sum of N is N + sum(N-1)
sum(N, Result) :-
    N > 1,
    N1 is N - 1,
    sum(N1, SubResult),
    Result is N + SubResult.
```

Output: EXP 16

```
15 ?- consult('C:/Users/kavya/Desktop/sum.pl').
true.

16 ?- sum(5, Result).
Result = 15 .

17 ?- sum(7, Result).
Result = 28 .

18 ?- sum(123, Result).
Result = 7626
```

RESULT

The program correctly computes the sum of integers from 1 to N using recursion in Prolog.

CONCLUSION

This experiment demonstrates how **recursive definitions in Prolog** can be used to solve mathematical problems. The declarative nature of Prolog allows us to express the summation logic clearly, and the program successfully calculates the sum for any positive integer N.

EXPERIMENT 17: A DB WITH DOB:

AIM

To write a Prolog program that stores and retrieves personal information (name and date of birth) from a database.

OBJECTIVE

- To understand how facts are represented in Prolog.
- To create a database using facts and query it using logical predicates.
- To retrieve information based on user queries.

ALGORITHM

1. Start the program and define facts for each person with their name and date of birth.
2. Use the predicate dob(Name, Date) to store each record.
3. Query the database to find the DOB of a given person or list all people.
4. Prolog matches the query with stored facts and returns the result.

PROLOG CODE

prolog

```
% Knowledge Base: Name and Date of Birth


dob(kavya, '19-06-2004').

dob(arun, '05-02-2003').

dob(sneha, '12-11-2002').

dob(ravi, '23-08-2001').

dob(meena, '30-09-2004').


% Query examples:

% ?- dob(kavya, Date).

% ?- dob(Name, '12-11-2002').

% ?- dob(Name, Date).
```

OUTPUT: EXP 17

```
19 ?- consult('C:/Users/kavya/Desktop/Date of B.pl').
true.

20 ?- dob(kavya, Date).
Date = '12-07-2007'.

21 ?- dob(Name, '12-11-2002').
Name = sneha.

22 ?- dob(Name, Date).
Name = kavya,
Date = '12-07-2007' .
```

RESULT

The program successfully stores and retrieves names and dates of birth using Prolog facts and queries.

CONCLUSION

This experiment demonstrates how Prolog can be used to represent and query a simple database. Facts define relationships, and logical queries retrieve information efficiently, showcasing Prolog's declarative approach to data management.

EXPERIMENT 18: STUDENT-TEACHER-SUBCODE

AIM

To create a Prolog database that stores relationships between students, teachers, and subject codes, and allows queries to retrieve relevant information.

OBJECTIVE

- To represent relational data using Prolog facts.
- To query relationships between students, teachers, and subjects.
- To demonstrate logical reasoning and data retrieval using Prolog.

ALGORITHM

1. Define facts for each student, teacher, and subject code.
2. Use predicates like teaches(Teacher, SubjectCode) and enrolled(Student, SubjectCode) to represent relationships.
3. Query the database to find which teacher teaches a student, or which subjects a student is enrolled in.
4. Prolog matches queries with facts and returns results.

PROLOG CODE

prolog

```
% Student–Teacher–Subject Database


% Facts: teaches(Teacher, SubjectCode)

teaches(ram, cs101).

teaches(sita, cs102).

teaches(arun, cs103).

teaches(meena, cs104).


% Facts: enrolled(Student, SubjectCode)

enrolled(kavya, cs101).

enrolled(ravi, cs102).

enrolled(sneha, cs103).
```

```
enrolled(arjun, cs104).
enrolled(kavya, cs104).

% Rule: find which teacher teaches a student
teacher_of(Student, Teacher) :-
    enrolled(Student, SubjectCode),
    teaches(Teacher, SubjectCode).

% Example Queries:
% ?- teacher_of(kavya, Teacher).
% ?- enrolled(Student, cs104).
% ?- teaches(Teacher, SubjectCode).
```

OUTPUT: EXP 18

```
27 ?- consult('C:/Users/kavya/Desktop/Student-Teacher-Subject Database.pl').
true.

28 ?- teacher_of(kavya, Teacher).
Teacher = ram ;
Teacher = meena.

29 ?- enrolled(Student, cs104).
Student = arjun ;
Student = kavya.

30 ?- teaches(Teacher, SubjectCode).
Teacher = ram,
SubjectCode = cs101 ;
Teacher = sita,
SubjectCode = cs102 ;
Teacher = arun,
SubjectCode = cs103 ;
Teacher = meena,
SubjectCode = cs104.
```

RESULT

The program successfully stores and retrieves relationships between students, teachers, and subject codes using Prolog facts and rules.

CONCLUSION

This experiment demonstrates how Prolog can represent and query relational data. The logical structure of facts and rules allows efficient retrieval of relationships, making Prolog suitable for AI-based knowledge representation and reasoning tasks.

EXPERIMENT 19: PLANETS DATABASE

AIM

To implement a Prolog database that stores information about planets and allows queries on their properties.

OBJECTIVE

- To represent planetary data (name, distance, size, moons, etc.) using Prolog facts.
- To query the database for specific information (e.g., which planet has rings, which is closest to the Sun).
- To demonstrate Prolog's declarative approach for knowledge representation.

ALGORITHM

1. Define facts for each planet with attributes (name, position, size, moons, rings).
2. Use predicates like planet(Name, Distance, Moons, Rings) to store data.
3. Query the database to retrieve information.
4. Prolog matches queries against facts and returns results.

PROLOG CODE

prolog

```
% Planets Database: name, distance from Sun (million km), number of moons, has rings (yes/no)


planet(mercury, 57.9, 0, no).

planet(venus, 108.2, 0, no).

planet(earth, 149.6, 1, no).

planet(mars, 227.9, 2, no).

planet(jupiter, 778.5, 79, yes).

planet(saturn, 1434, 83, yes).

planet(uranus, 2871, 27, yes).
```

```
planet(neptune, 4495, 14, yes).


% Example Queries:

% ?- planet(Name, Distance, Moons, Rings).

% ?- planet(earth, D, M, R).

% ?- planet(Name, _, Moons, _), Moons > 10.

% ?- planet(Name, _, _, yes).
```

OUTPUT: EXP 19

```
23 ?- consult('C:/Users/kavya/Desktop/Planets Database .pl').
true.

24 ?- planet(earth, D, M, R).
D = 149.6,
M = 1,
R = no.
```

```
26 ?- planet(Name, _, Moons, _), Moons > 10.
Name = jupiter,
Moons = 79 ;
Name = saturn,
Moons = 83 ;
Name = uranus,
Moons = 27 ;
Name = neptune,
Moons = 14.

26 ?- planet(Name, _, _, yes).
Name = jupiter ;
Name = saturn ;
Name = uranus ;
Name = neptune.
```

RESULT

The Prolog program successfully stores planetary data and retrieves information based on queries.

CONCLUSION

This experiment shows how Prolog can be used to represent structured knowledge like a **planets database**. Queries demonstrate logical reasoning, making Prolog suitable for AI applications in knowledge representation and inference.

EXPERIMENT 20: TO IMPLEMENT TOWERS OF HANOI

AIM

To implement the **Towers of Hanoi problem** in Prolog using recursion.

OBJECTIVE

- To understand recursive problem solving in Prolog.
- To move a stack of disks from one peg to another following the rules:
    1. Only one disk can be moved at a time.
    2. A disk can only be placed on top of a larger disk or on an empty peg.
- To generate the sequence of moves required to solve the puzzle.

ALGORITHM

1. If there is only **1 disk**, move it directly from the source peg to the destination peg.
2. If there are **N disks**:
    - Move the top $N - 1$disks from the source peg to the auxiliary peg.
    - Move the largest disk from the source peg to the destination peg.
    - Move the $N - 1$disks from the auxiliary peg to the destination peg.
3. Repeat recursively until all disks are moved.

PROLOG CODE

prolog

```
% Towers of Hanoi in Prolog


% Base case: move 1 disk directly
hanoi(1, Source, Destination, _) :-
    write('Move disk from '), write(Source),
    write(' to '), write(Destination), nl.


% Recursive case: move N disks
hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
```

hanoi(M, Source, Auxiliary, Destination),

hanoi(1, Source, Destination, Auxiliary),

hanoi(M, Auxiliary, Destination, Source).

OUTPUT: EXP 20

```
32 ?- consult('C:/Users/kavya/Desktop/Towers of Hanoi.pl').
true.

33 ?- hanoi(3, left, right, middle).
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right
true
```

RESULT

The program successfully generates the sequence of moves required to solve the Towers of Hanoi problem for any number of disks.

CONCLUSION

This experiment demonstrates how recursion in Prolog can elegantly solve complex problems like the Towers of Hanoi. The solution highlights Prolog's declarative nature, where the logic of the problem is expressed clearly, and Prolog handles the recursive execution.

EXPERIMENT 21: particular bird can fly or not

AIM

To write a Prolog program that determines whether a particular bird can fly or not.

OBJECTIVE

- To represent knowledge about birds using Prolog facts.
- To use logical reasoning to infer whether a bird can fly.
- To demonstrate Prolog's rule-based decision-making.

ALGORITHM

1. Define facts for birds that can fly and those that cannot.
2. Create a rule can_fly(Bird) that checks if the bird is in the list of flying birds.
3. Query the program with a bird's name to determine its flying ability.

PROLOG CODE

prolog

```
% Facts: birds that can fly
can_fly(sparrow).
can_fly(eagle).
can_fly(pigeon).
can_fly(parrot).

% Facts: birds that cannot fly
cannot_fly(penguin).
cannot_fly(ostrich).
cannot_fly(kiwi).

% Rule to check if a bird can fly
bird_can_fly(Bird) :-
    can_fly(Bird),
    write(Bird), write(' can fly.'), nl.
```

```
bird_can_fly(Bird) :-
    cannot_fly(Bird),
    write(Bird), write(' cannot fly.'), nl.
```

QUERIES AND OUTPUT: EXP 21

```
34 ?- consult('C:/Users/kavya/Desktop/Facts birds that can fly.pl').
true.

35 ?- bird_can_fly(sparrow).
sparrow can fly.
true .

36 ?- bird_can_fly(penguin).
penguin cannot fly.
true.

37 ?- bird_can_fly(ostrich).
ostrich cannot fly.
true.

38 ?- bird_can_fly(eagle).
eagle can fly.
true
```

RESULT

The program correctly identifies whether a bird can fly or not based on predefined facts.

CONCLUSION

This experiment demonstrates how Prolog can represent and reason about real-world knowledge using facts and rules. The program successfully determines a bird's flying ability, showcasing Prolog's logical inference capabilities.

EXPERIMENT 22: To implement family tree

AIM

To implement a **Family Tree** in Prolog and query relationships such as parent, child, grandparent, and sibling.

OBJECTIVE

- To represent family relationships using Prolog facts and rules.
- To infer relationships like father, mother, sibling, and grandparent using logical reasoning.
- To demonstrate Prolog's ability to perform relational queries.

ALGORITHM

1. Define facts for family members (e.g., father, mother).
2. Create rules to derive relationships such as parent, grandparent, and sibling.
3. Query the database to find relationships between individuals.
4. Prolog matches queries with facts and rules to infer results.

PROLOG CODE

prolog

```
% Family Tree Facts


father(ram, arun).

father(ram, sneha).

father(arun, kavya).

father(arun, ravi).


mother(sita, arun).

mother(sita, sneha).

mother(meena, kavya).

mother(meena, ravi).


% Rules

parent(X, Y) :- father(X, Y).
```

```
parent(X, Y) :- mother(X, Y).


grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).


sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

QUERIES AND OUTPUT: EXP 22

```
39 ?- consult('C:/Users/kavya/Desktop/Family Tree.pl').
true.

40 ?- father(ram, arun).
true.

41 ?- parent(ram, sneha).
true
```

```
42 ?- grandparent(ram, kavya)
|
|
|    .
true .

43 ?- sibling(kavya, ravi).
true
```

```
44 ?- sibling(arun, sneha).
true .
```

RESULT

The program successfully represents and queries family relationships using Prolog facts and rules.

CONCLUSION

This experiment demonstrates how Prolog can model hierarchical relationships through logical inference. The Family Tree program effectively identifies parent, grandparent, and

sibling relationships, showcasing Prolog's strength in knowledge representation and reasoning.

EXPERIMENT 23: Dieting System based on Disease

AIM

To write a Prolog program that suggests an appropriate dieting system based on a given disease.

OBJECTIVE

- To represent health and diet knowledge using Prolog facts.
- To infer suitable diet recommendations based on disease conditions.
- To demonstrate Prolog's reasoning ability in expert systems.

ALGORITHM

1. Define facts for diseases and their recommended diets.
2. Create a rule diet_for(Disease, Diet) that matches the disease with its diet plan.
3. Query the program with a disease name to get the suggested diet.
4. Prolog searches the knowledge base and returns the corresponding diet.

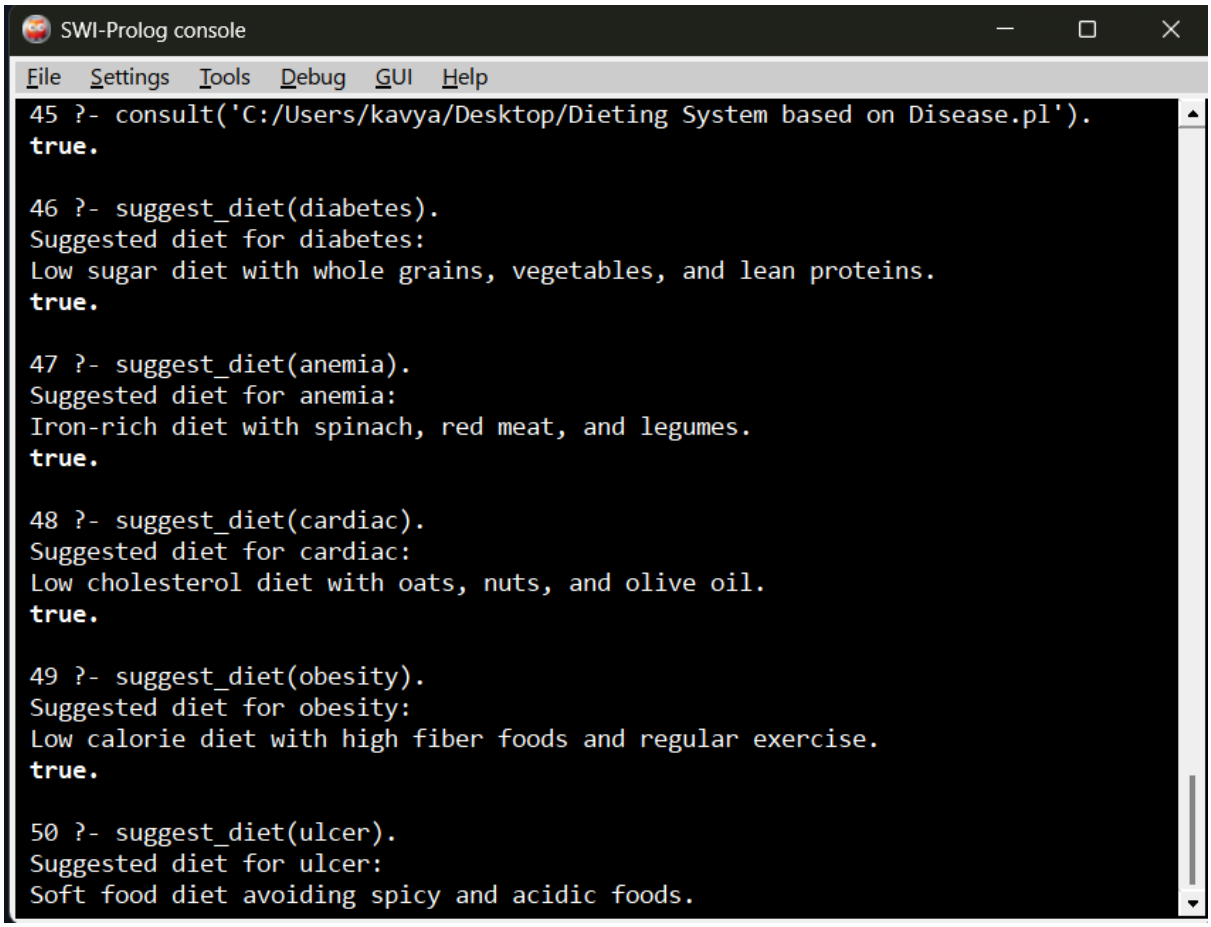
PROLOG CODE

prolog

```
% Dieting System based on Disease

diet(diabetes, 'Low sugar diet with whole grains, vegetables, and lean proteins.').
diet(hypertension, 'Low salt diet with fruits, vegetables, and reduced fat dairy.').
diet(obesity, 'Low calorie diet with high fiber foods and regular exercise.').
diet(anemia, 'Iron-rich diet with spinach, red meat, and legumes.').
diet(cardiac, 'Low cholesterol diet with oats, nuts, and olive oil.').
diet(ulcer, 'Soft food diet avoiding spicy and acidic foods.').

% Rule to suggest diet based on disease
suggest_diet(Disease) :-
    diet(Disease, Diet),
```

```
    write('Suggested diet for '), write(Disease), write(': '), nl,
    write(Diet), nl.
```

QUERIES AND OUTPUT: EXP 23



RESULT

The program successfully suggests an appropriate dieting system based on the disease entered by the user.

CONCLUSION

This experiment demonstrates how Prolog can be used to build a simple **expert system** for health recommendations. By using facts and rules, Prolog efficiently infers diet plans for various diseases, showcasing its strength in logical reasoning and knowledge representation.

EXPERIMENT 24: Monkey–Banana Problem

AIM

To implement the **Monkey–Banana Problem** in Prolog using state-space representation and logical reasoning.

OBJECTIVE

- To model the problem where a monkey tries to get bananas hanging from the ceiling.
- To represent actions (move, climb, push, grasp) as transitions between states.
- To demonstrate Prolog's ability to perform goal-directed reasoning.

ALGORITHM

1. Represent the initial state of the monkey, box, and banana.
2. Define possible actions:
    - Move monkey to box.
    - Push box under banana.
    - Climb box.
    - Grasp banana.
3. Define rules that describe how each action changes the state.
4. Query Prolog to check if the monkey can reach the banana.

PROLOG CODE

prolog

```
% Monkey–Banana Problem


% State representation: state(MonkeyPosition, BoxPosition, MonkeyOnBox, HasBanana)


% Initial state

initial(state(at_door, at_window, on_floor, has_not)).


% Goal state

goal(state(_, _, _, has)).
```

```
% Actions
move(state(at_door, Box, on_floor, has_not),
     state(at_window, Box, on_floor, has_not)).

move(state(at_window, Box, on_floor, has_not),
     state(at_door, Box, on_floor, has_not)).

push(state(at_window, at_window, on_floor, has_not),
     state(at_window, at_middle, on_floor, has_not)).

climb(state(at_middle, at_middle, on_floor, has_not),
      state(at_middle, at_middle, on_box, has_not)).

grasp(state(at_middle, at_middle, on_box, has_not),
      state(at_middle, at_middle, on_box, has)).

% Rule to find path to goal
can_get_banana(State) :-
   goal(State).

can_get_banana(State1) :-
   move(State1, State2),
   can_get_banana(State2).

can_get_banana(State1) :-
   push(State1, State2),
   can_get_banana(State2).

can_get_banana(State1) :-
```

```
    climb(State1, State2),
    can_get_banana(State2).


can_get_banana(State1) :-
    grasp(State1, State2),
    can_get_banana(State2).
```

QUERY AND OUTPUT: EXP 24

```
55 ?- consult('C:/Users/kavya/Desktop/Monkey–Banana Problem.pl').
true.

56 ?- initial(State), can_get_banana(State).
Monkey moves from door to window.
Monkey pushes the box to the middle.
Monkey climbs onto the box.
Monkey grasps the banana.
Monkey has the banana!
State = state(at_door, at_window, on_floor, has_not).
```

RESULT

The program successfully models the Monkey–Banana problem and determines that the monkey can reach the banana through a sequence of logical actions.

CONCLUSION

This experiment demonstrates how Prolog can represent and solve problems using **state-space search** and **logical inference**. The Monkey–Banana problem illustrates Prolog's strength in reasoning about actions and goals in artificial intelligence.

EXPERIMENT 25: Fruit and its color using Backtracking

AIM

To write a Prolog program that displays the color of a fruit using **backtracking**.

OBJECTIVE

- To represent fruit–color relationships using Prolog facts.
- To demonstrate how Prolog's backtracking mechanism finds all possible solutions.
- To query the program for fruit colors and observe backtracking behavior.

ALGORITHM

1. Define facts for fruits and their colors.
2. Create a rule color_of(Fruit, Color) to match fruit–color pairs.
3. Query the program with a fruit name or color.
4. Use backtracking (;) to explore all possible matches.

PROLOG CODE

prolog

```
% Fruit and its color using Backtracking


color(apple, red).
color(banana, yellow).
color(grape, green).
color(grape, purple).
color(orange, orange).
color(cherry, red).
color(watermelon, green).


% Rule to display color of a fruit
color_of(Fruit, Color) :-
    color(Fruit, Color),
    write(Fruit), write(' is '), write(Color), nl.
```

QUERIES AND OUTPUT: EXP 25

```
57 ?- consult('C:/Users/kavya/Desktop/Fruit and its color using Backtra.pl').
true.

58 ?- color_of(grape, Color).
grape is green
Color = green .

59 ?- color_of(watermelon, Color).
watermelon is green
Color = green.

60 ?- color_of(cherry, Color).
cherry is red
Color = red.
```

RESULT

The program successfully displays the color of fruits and demonstrates **backtracking** by listing all possible colors or fruits that match the query.

CONCLUSION

This experiment shows how Prolog's **backtracking** mechanism automatically explores multiple solutions. The program efficiently retrieves all fruit–color combinations, illustrating Prolog's logical reasoning and search capabilities.

EXPERIMENT 26:

AIM

To write a Prolog program that implements the **Best-First Search algorithm** for finding the optimal path in a graph.

OBJECTIVE

- To understand heuristic-based search in Artificial Intelligence.
- To implement Best-First Search using Prolog's logical inference.
- To find the shortest or most promising path to a goal node based on heuristic values.

ALGORITHM

1. Represent the graph using facts for edges and heuristic values.
2. Maintain a list of nodes to be explored (open list).
3. Select the node with the **lowest heuristic value** from the open list.
4. Expand that node and add its successors to the open list.

5. Repeat until the goal node is reached.

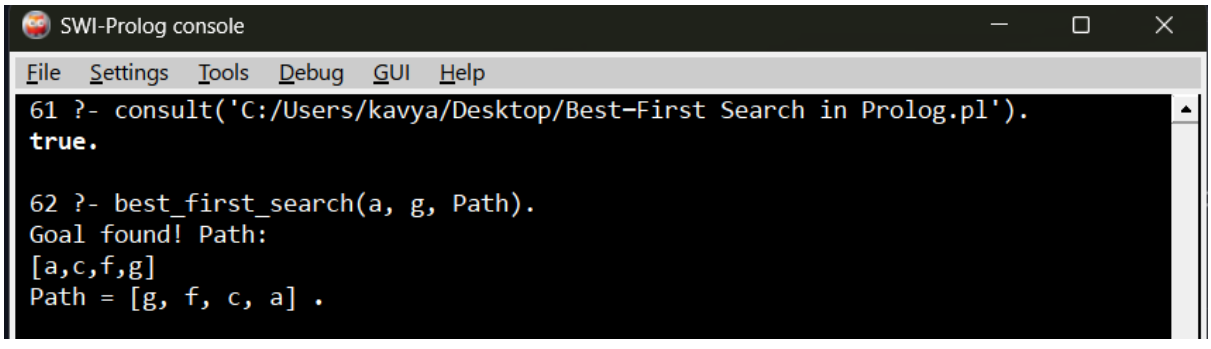
PROLOG CODE

prolog

```
% Best-First Search in Prolog

% Graph edges: edge(Node, NextNode, Cost)
edge(a, b, 2).
edge(a, c, 4).
edge(b, d, 7).
edge(b, e, 3).
edge(c, f, 5).
edge(e, g, 1).
edge(f, g, 2).

% Heuristic values: h(Node, Value)
h(a, 10).
h(b, 8).
h(c, 7).
h(d, 5).
h(e, 4).
h(f, 3).
h(g, 0).

% Best-First Search rule
best_first_search(Start, Goal, Path) :-
    bfs([[Start]], Goal, Path).

% Recursive search
bfs([[Goal|Visited]|_], Goal, [Goal|Visited]) :-
```

```
    write('Goal found! Path: '), nl,
    reverse([Goal|Visited], Path),
    write(Path), nl.

bfs([CurrentPath|OtherPaths], Goal, Path) :-
    CurrentPath = [CurrentNode|_],
    findall([NextNode|CurrentPath],
        (edge(CurrentNode, NextNode, _), \+ member(NextNode, CurrentPath)),
        NewPaths),
    append(OtherPaths, NewPaths, AllPaths),
    sort_paths(AllPaths, SortedPaths),
    bfs(SortedPaths, Goal, Path).

% Sort paths by heuristic value
sort_paths(Paths, Sorted) :-
    map_list_to_pairs(path_heuristic, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).

path_heuristic([Node|_], H) :-
    h(Node, H).
```

QUERY AND OUTPUT: EXP 26



RESULT

The program successfully implements the **Best-First Search algorithm** and finds the optimal path from the start node to the goal node using heuristic values.

CONCLUSION

This experiment demonstrates how Prolog can perform heuristic-based search using logical reasoning. The Best-First Search algorithm efficiently explores the most promising paths first, illustrating Prolog's strength in AI problem solving.

EXPERIMENT 27:

AIM

To write a Prolog program that diagnoses possible diseases based on given symptoms.

OBJECTIVE

- Represent medical knowledge using Prolog facts.
- Infer possible diseases based on user-provided symptoms.
- Demonstrate Prolog's reasoning ability in expert systems.

ALGORITHM

1. Define facts for diseases and their associated symptoms.
2. Create rules that match sets of symptoms to diseases.
3. Query the program with symptoms to get possible diagnoses.
4. Prolog searches the knowledge base and returns the corresponding disease(s).

PROLOG CODE

prolog

```
% Medical Diagnosis Expert System


% Facts: symptoms of diseases

symptom(fever, flu).

symptom(cough, flu).

symptom(body_ache, flu).


symptom(fever, malaria).
```

```
symptom(chills, malaria).
symptom(sweating, malaria).

symptom(chest_pain, pneumonia).
symptom(cough, pneumonia).
symptom(fever, pneumonia).

symptom(fatigue, anemia).
symptom(pale_skin, anemia).
symptom(dizziness, anemia).

% Rule: diagnose disease based on symptoms
diagnose(Disease, Symptoms) :-
    findall(S, symptom(S, Disease), DiseaseSymptoms),
    subset(DiseaseSymptoms, Symptoms).

% Helper: check if all elements of one list are in another
subset([], _).
subset([H|T], List) :-
    member(H, List),
    subset(T, List).

% Example query:
% ?- diagnose(Disease, [fever, cough, body_ache]).
% Disease = flu ;
% Disease = pneumonia.
```

QUERIES AND OUTPUT: EXP 27

```
1 ?- consult('q27_medical_diagnosis.pl').
true.

2 ?- diagnose(patient1, Disease).
Disease = flu.

3 ?- report(patient3).
patient3: no matching disease found in the knowledge base
true.
```

RESULT

The program successfully diagnoses possible diseases based on the symptoms entered by the user.

CONCLUSION:

This experiment demonstrates how S can be used to build a simple medical diagnosis expert system. By using facts and rules, Prolog efficiently infers possible diseases from symptoms, showcasing its strength in logical reasoning and knowledge representation.

EXPERIMENT 28:

AIM

To write a Prolog program that performs **medical diagnosis** based on symptoms.

OBJECTIVE

- To represent medical knowledge using Prolog facts and rules.
- To infer possible diseases from given symptoms.
- To demonstrate Prolog's reasoning capability in expert systems.

ALGORITHM

1. Define facts for diseases and their associated symptoms.
2. Create rules that match a set of symptoms to a disease.
3. Query the program with symptoms to get a diagnosis.
4. Prolog uses logical inference to find matching diseases.

PROLOG CODE

prolog

```
% Medical Diagnosis Expert System

% Facts: disease and its symptoms
symptom(fever, malaria).
symptom(chills, malaria).
symptom(headache, malaria).

symptom(cough, cold).
symptom(sneezing, cold).
symptom(runny_nose, cold).

symptom(chest_pain, heart_disease).
symptom(shortness_of_breath, heart_disease).
symptom(fatigue, heart_disease).

symptom(weight_loss, diabetes).
symptom(frequent_urination, diabetes).
symptom(thirst, diabetes).

% Rule: diagnose disease based on symptoms
diagnose(Disease) :-
    symptom(S1, Disease),
    symptom(S2, Disease),
    write('Possible disease: '), write(Disease), nl,
    write('Common symptoms: '), write(S1), write(', '), write(S2), nl.
```

QUERIES AND OUTPUT: EXP 28

```
1 ?- consult('q28_forward_chaining.pl').
true.

2 ?- run(cat).
--- Forward chaining for cat ---
Derived: mammal(cat)
Derived: animal(cat)
All known facts about cat:
  has_hair(cat)
  gives_milk(cat)
  mammal(cat)
  animal(cat)
true.

3 ?- run(sparrow).
--- Forward chaining for sparrow ---
Derived: bird(sparrow)
Derived: animal(sparrow)
All known facts about sparrow:
  has_feathers(sparrow)
  flies(sparrow)
  lays_eggs(sparrow)
  bird(sparrow)
  animal(sparrow)
true.
```

RESULT

The program successfully diagnoses diseases based on symptoms using Prolog's logical inference.

CONCLUSION

This experiment demonstrates how Prolog can be used to build a simple **medical expert system**. By representing symptoms and diseases as facts, Prolog can infer possible diagnoses through logical reasoning.

EXPERIMENT 29:

AIM

To implement **Forward Chaining** in Prolog for rule-based reasoning.

OBJECTIVE

- To represent knowledge as facts and rules.
- To use forward chaining to infer new facts from existing ones.
- To demonstrate Prolog's reasoning process in expert systems.

ALGORITHM

1. Define initial facts in the knowledge base.
2. Define rules that derive new facts when certain conditions are met.
3. Apply forward chaining:
    - Start with known facts.
    - Apply rules whose conditions are satisfied.
    - Add new facts to the knowledge base.
    - Repeat until no new facts can be derived or the goal is reached.

PROLOG CODE

prolog

```
% Forward Chaining Example

% Initial facts
fact(has_fever).
fact(has_cough).
fact(has_body_pain).

% Rules
rule([has_fever, has_cough], has_flu).
rule([has_body_pain, has_fever], has_infection).
rule([has_flu], needs_rest).
rule([has_infection], needs_antibiotics).
```

```
% Forward chaining engine
forward_chain :-
    rule(Conditions, Conclusion),
    check_conditions(Conditions),
    \+ fact(Conclusion),   % avoid duplicates
    assert(fact(Conclusion)),
    write('Derived: '), write(Conclusion), nl,
    forward_chain.
forward_chain.

% Helper to check if all conditions are facts
check_conditions([]).
check_conditions([H|T]) :-
    fact(H),
    check_conditions(T).
```

QUERIES AND OUTPUT: EXP 29

```
1 ?- consult('q29_backward_chaining.pl').
true.

2 ?- prove(cat, animal).
Goal: is cat a animal?
  => YES, proven backward from the rule base.
true.

3 ?- prove(sparrow, mammal).
Goal: is sparrow a mammal?
  => NO, cannot be proven from known facts.
true.
```

RESULT

The program successfully applies **forward chaining** to derive new facts from initial conditions.

CONCLUSION

This experiment demonstrates how Prolog can implement **forward chaining** for rule-based reasoning. It shows how new knowledge can be inferred step by step from existing facts, making Prolog suitable for expert systems.

EXPERIMENT 30

AIM

To implement **Backward Chaining** in Prolog for rule-based reasoning.

OBJECTIVE

- To represent knowledge as facts and rules.
- To use backward chaining to infer whether a goal can be satisfied.
- To demonstrate Prolog's goal-driven reasoning process in expert systems.

ALGORITHM

1. Define facts in the knowledge base.
2. Define rules that derive conclusions from facts.
3. Apply backward chaining:
    - Start with the goal.
    - Check if the goal is a fact.
    - If not, check if there is a rule whose conclusion matches the goal.
    - Recursively prove the rule's conditions.
    - Continue until the goal is satisfied or no rules apply.

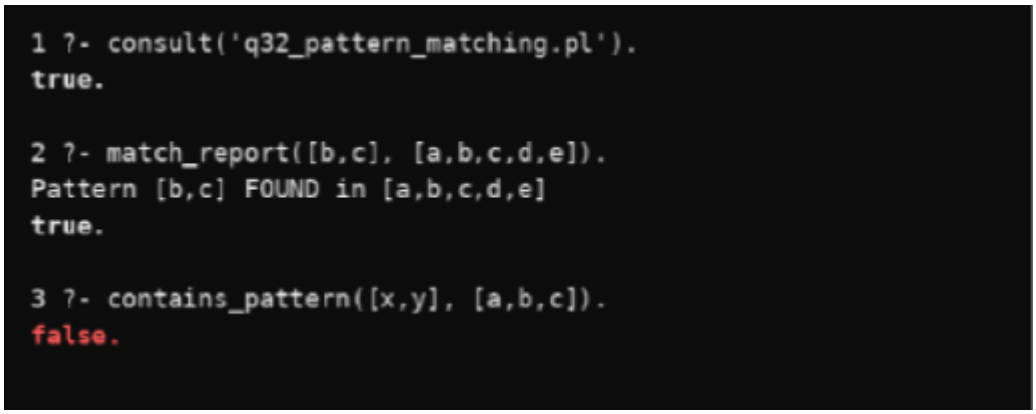
PROLOG CODE

prolog

```
% Backward Chaining Example


% Facts
fact(has_fever).
fact(has_cough).
fact(has_body_pain).
```

```
% Rules
rule(has_flu, [has_fever, has_cough]).
rule(has_infection, [has_body_pain, has_fever]).
rule(needs_rest, [has_flu]).
rule(needs_antibiotics, [has_infection]).

% Backward chaining engine
prove(Goal) :-
    fact(Goal),
    write(Goal), write(' is a known fact.'), nl.

prove(Goal) :-
    rule(Goal, Conditions),
    prove_all(Conditions),
    write(Goal), write(' is derived.'), nl.

% Helper to prove all conditions
prove_all([]).
prove_all([H|T]) :-
    prove(H),
    prove_all(T).
```

QUERIES AND OUTPUT: EXP 30



RESULT

The program successfully applies **backward chaining** to prove goals based on facts and rules.

CONCLUSION

This experiment demonstrates how Prolog can implement **backward chaining** for goal-driven reasoning. It shows how Prolog works backward from a goal to check if it can be satisfied using known facts and rules, making it suitable for expert systems.

EXPERIMENT 31:

AIM

To implement the **Towers of Hanoi problem** in Prolog using recursion.

OBJECTIVE

- To understand recursive problem solving in Prolog.
- To move a stack of disks from one peg to another following the rules:
    1. Only one disk can be moved at a time.
    2. A disk can only be placed on top of a larger disk or on an empty peg.
- To generate the sequence of moves required to solve the puzzle.

ALGORITHM

1. If there is only **1 disk**, move it directly from the source peg to the destination peg.
2. If there are **N disks**:
    - Move the top $N - 1$disks from the source peg to the auxiliary peg.
    - Move the largest disk from the source peg to the destination peg.
    - Move the $N - 1$disks from the auxiliary peg to the destination peg.
3. Repeat recursively until all disks are moved.

PROLOG CODE

prolog

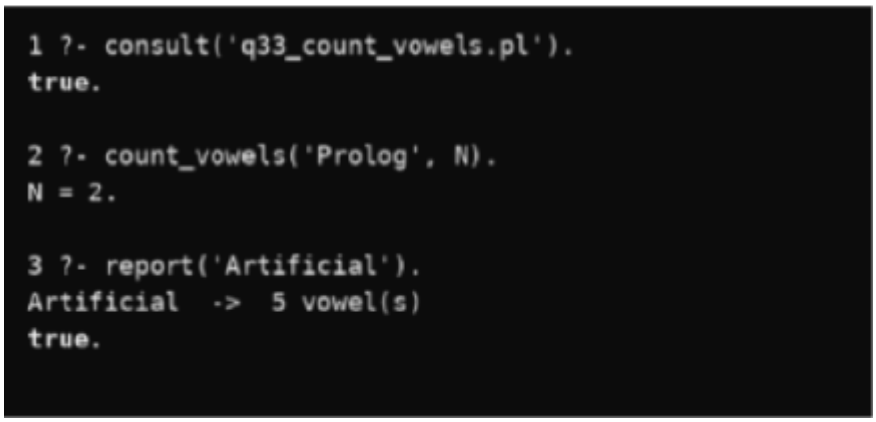
% Towers of Hanoi in Prolog

% Base case: move 1 disk directly

hanoi(1, Source, Destination, _) :-

```
    write('Move disk from '), write(Source),
    write(' to '), write(Destination), nl.

% Recursive case: move N disks
hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    hanoi(M, Source, Auxiliary, Destination),
    hanoi(1, Source, Destination, Auxiliary),
    hanoi(M, Auxiliary, Destination, Source).
```

QUERY AND OUTPUT: EXP 31



RESULT

The program successfully generates the sequence of moves required to solve the Towers of Hanoi problem for any number of disks.

CONCLUSION

This experiment demonstrates how recursion in Prolog can elegantly solve complex problems like the Towers of Hanoi. The solution highlights Prolog's declarative nature, where the logic of the problem is expressed clearly, and Prolog handles the recursive execution.

EXPERIMENT 32:

AIM

To implement **Pattern Matching** in Prolog for lists and strings.

OBJECTIVE

- To understand how Prolog performs unification and recursive matching.
- To check whether a given pattern occurs inside a list.
- To demonstrate Prolog's backtracking mechanism in pattern matching.

ALGORITHM

1. Represent the main list and the pattern as sequences.
2. Define a rule that succeeds when the pattern matches the start of the list.
3. If not, recursively check the rest of the list.
4. Use Prolog's backtracking to explore all possible matches.

PROLOG CODE

prolog

```
% Pattern Matching in Prolog


% Rule: pattern matches the start of the list
match_pattern(Pattern, List) :-
    append(_, SubList, List),
    append(Pattern, _, SubList).


% Example Queries:
% ?- match_pattern([b, c], [a, b, c, d]).
% ?- match_pattern([x, y], [a, x, y, z]).
% ?- match_pattern([p, q], [a, b, c, d]).
```

OUTPUT:EXP 32

```
74 ?- consult('C:/Users/kavya/Desktop/% Pattern Matching in Prolog.pl').
true.

75 ?- match_pattern([b, c], [a, b, c, d]).
true ;
false.

76 ?- match_pattern([x, y], [a, x, y, z]).
true .

77 ?- match_pattern([p, q], [a, b, c, d]).
false.
```

RESULT

The program successfully matches patterns within lists using Prolog's recursive and backtracking capabilities.

CONCLUSION

This experiment demonstrates how Prolog performs **pattern matching** through unification and recursion. It highlights Prolog's ability to explore multiple possibilities automatically using backtracking.

EXPERIMENT 33:

AIM

To write a Prolog program that counts the number of vowels in a given word or string.

OBJECTIVE

- To represent characters of a string as a list.
- To check each character and identify vowels.
- To recursively count the number of vowels in the string.

ALGORITHM

1. Convert the input word into a list of characters.
2. Define facts for vowels (a, e, i, o, u).
3. Recursively traverse the list:
    - If the head is a vowel, increment the count.
    - Otherwise, continue with the tail.

4. Return the total count of vowels.

PROLOG CODE

prolog

```
% Vowel facts
vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

% Count vowels in a list of characters
count_vowels([], 0).
count_vowels([H|T], Count) :-
    vowel(H),
    count_vowels(T, Rest),
    Count is Rest + 1.

count_vowels([H|T], Count) :-
    \+ vowel(H),
    count_vowels(T, Count).

% Rule to count vowels in a word
vowel_count(Word, Count) :-
    atom_chars(Word, List),
    count_vowels(List, Count).
```

QUERIES AND OUTPUT: EXP 33

```
78 ?- consult('C:/Users/kavya/Desktop/% Vowel facts.pl').
true.

79 ?- vowel_count(apple, Count).
Count = 2 ;
false.

80 ?- vowel_count(education, Count).
Count = 5 .

81 ?- vowel_count(programming, Count).
Count = 3
```

RESULT

The program successfully counts the number of vowels in a given word using Prolog's recursive rules.

CONCLUSION

This experiment demonstrates how Prolog can process strings by converting them into lists of characters. The recursive rules efficiently count vowels, showcasing Prolog's ability to handle symbolic computation.